

Fig. 2 This transition graph is a DFA for the regular expression $(ab)^*abb$

such that the symbols along the path spell out x. So, the input string bbabb is accepted because of the path 0-1-4-5-6-1-4-5-6-7-8-9-10.

[optional] Q. Find out three strings that are not accepted by the NFA for L.

A DFA is a special case of an NFA where:

1. There are no moves on input ϵ , and
2. For each state s and input symbol a, there is exactly one edge out of s labeled a.

This is the reason why a DFA shares the same notation as an NFA.

While the NFA is an abstract representation of an algorithm to recognize the strings of a certain language, the DFA is a simple and concrete algorithm for recognizing strings. The DFA for L is shown in Fig. 2

The input string abbabb will be accepted by this DFA because of the path 0-1-2-3-1-2-3. Note that a string will be accepted by the DFA only if it ends on the end state (3).

Deterministic Finite Automata (DFA) is the most standard variant and was invented to model and study real-life scenarios that the Turing Machine (https://en.wikipedia.org/wiki/Turing_machine) was unable to demonstrate. Since the DFA is used to implement or simulate when building lexical analyzers, it is fortunate that every regular expression or NFA sharing the same language can be converted into a DFA, which is beyond the scope of this article.

[optional] We can represent either an NFA or DFA by a transition graph, where there is an edge labeled 'a' from state 's' to state

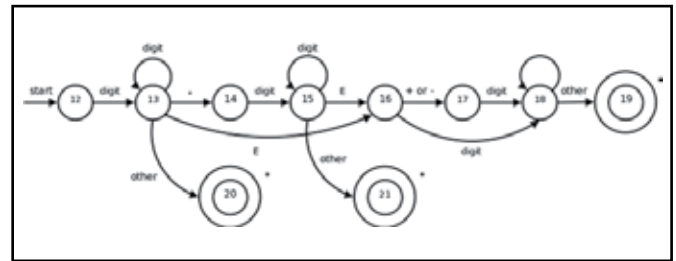


Fig. 4. This is an example of how the transition diagram looks like in automata theory

't' if and only if 't' is one of the next states for state 's' and input 'a'. In this graph, as shown in Fig. 3 for language L,

- a. The same symbol can label edges from one state to many different states, and
- b. An edge may be labeled by ϵ , instead of or in addition to symbols from the input alphabet.

For searching *.java files, a DFA can prove to be very useful if the filename is composed of ASCII characters (real language) only.

Automata simulators are used to learn and research automata theory. An automata simulator takes as input the description of an automaton and then simulates its working for an arbitrary input string. The description of the automaton can be entered in several ways. An automaton can be defined in a symbolic language or its specification may be entered in a predesigned form or its transition diagram may be drawn by clicking and dragging the mouse. Well known automata simulators include Turing's World, JFLAP, VAS, TAGS and SimStudio

In conclusion, each model in the automata theory plays important roles in several applied areas. Finite Automata are used in text processing, compilers and hardware design, and Context-Free Grammar (CFG) [http://en.wikipedia.org/wiki/Context-free_grammar] is used in programming languages and artificial intelligence and also for study in human languages.

[optional] Let's see if you can figure out what this transition diagram accepts as a language.

Answer: Unsigned numbers (with E used as exponent for base 10).>

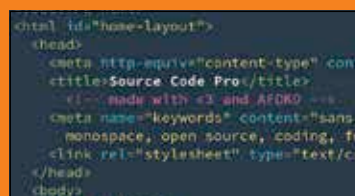
*the source code



DirectX 12 programming

>>The biggest piece of good news to PC gaming is Microsoft's decision to enable low-level GPU access. This will percolate down to consoles and mobile GPUs, too.

<http://dgit.in/NOvYkC>



Facebook streamlines Hack language

>>Facebook's dedication to streamlining programming language Hack (used internally) by making it open source

<http://dgit.in/1h5vTQN>



DARPA's contribution to open source software

>>While the Pentagon can be secretive about its operation, it's surprisingly open about the software projects it funded

<http://dgit.in/1iwWLSQ>